

DIGITAL TWIN CATALOG

Domain-Driven Design Document

Bounded contexts, shared language, aggregates and business rules of the digital twin catalog.

Project	Digital Twin Catalog (NARAGA 3D Catalog), reusable 3DCityDB template
Document	Domain-Driven Design
Version	0.1 draft for review, 4 October 2026
Author	Fairuz Akmal Pradana
Repository	github.com/fakmalpradana/naraga-twin-db

Contents

Introduction

Bounded contexts

Ubiquitous language

Aggregates and invariants

Domain events (documented, not event-sourced)

Decisions that shape the model

Introduction

Status: **draft v0.1 for review** · Date: 2026-10-03 · Related: [ERD](#), [Database schema](#), [SAD](#)

This repository is a **template**: the domain below is not specific to one city. Project-specific facts (OIKN) live only in `db/seeds/<project>/` .

Bounded contexts

Context	Responsibility	Owns	Store
Catalog	Register datasets, layers (theme × LOD), tilesets, import history; serve the frontend	dataset , layer , tileset , import_job , reference lists	schema catalog
City Model	Semantic 3D city objects (CityGML 2.0): buildings, parts, rooms, surfaces, attributes	all citydb.* tables (3DCityDB v5)	schema citydb (never modified, except one index)
Tile Publishing	Convert a layer to 3D Tiles and upload (convertwin → Cesium ion, or self-hosted)	tile build output, Cesium ion assets	files / Cesium ion
Administration & Audit	Who may edit; who changed what, when	users/groups, audit.change_log	schemas app , audit

Relationships: Catalog → City Model is **conformist read** (Catalog reads `citydb.feature` via the lineage tag; it never writes city objects). Tile Publishing is **downstream** of City Model and **reports back** to Catalog (writes `tileset`). Administration wraps every write in Catalog with a mandatory user name.

Ubiquitous language

Term	Meaning
Dataset	Data for one area from one source (e.g. <code>oikn</code> , <code>ikn_naraga</code>). Has a short unique code .
Theme	Subject of data: building, bridge, road, terrain, ... (reference list, extendable).
LOD	Level of Detail 0–4 as defined by CityGML 2.0.
Layer	One theme at one LOD of a dataset. The unit the frontend selects . Unique per (dataset, theme, LOD).
Tileset	One versioned 3D Tiles build of a layer. Exactly one may be active (served).
Import job	One file imported into the City Model (also the upload registry). A job can fill several layers.
Lineage tag	Text <code>dataset.theme.lodN</code> stored in <code>citydb.feature.lineage</code> on every imported object; the link between Catalog and City Model.
Stale	The data changed after the active tileset was built; tiles must be rebuilt.
Published	A layer visible to the frontend. Requires an active Ready tileset.
Editor / Viewer / Admin	Roles: edit catalog data / read-only / also delete & manage users.

Aggregates and invariants

Dataset is the aggregate root for Catalog. Layers and import jobs change only through a dataset context; tilesets only through their layer.

#	Invariant	Enforced by
I1	<code>dataset.code</code> is <code>[a-z0-9_]+</code> , unique, and frozen once objects carry its tag	CHECK, UNIQUE, trigger <code>dataset_guard</code>
I2	<code>LOD</code> $\in \{0..4\}$; theme must exist in <code>ref_theme</code>	FK to reference tables
I3	(dataset, theme, LOD) is unique	UNIQUE
I4	A layer's dataset/theme/LOD cannot change after data was imported into it	trigger <code>layer_guard</code>
I5	Layer status moves only: draft→validated→published→archived (plus validated→draft, published→validated, archived→draft, any non-published→archived)	trigger <code>layer_guard</code>
I6	draft→validated requires a succeeded import job with passed validation that filled this layer	trigger <code>layer_guard</code>
I7	Published ⇒ exactly one active, Ready tileset (checked at COMMIT so v1→v2 switch is atomic)	deferred constraint trigger + EXCLUDE
I8	At most one active tileset per layer	EXCLUDE (deferrable)
I9	Only a Ready tileset can be active; provider <code>cesium_ion</code> needs <code>ion_asset_id</code> , <code>self_hosted</code> needs <code>url</code>	CHECK
I10	Archiving a layer retires its tilesets; archiving a dataset archives its layers	cascade triggers
I11	Every write names a user ; <code>created_by/updated_by</code> come from the session, never from client input	trigger <code>stamp</code> , <code>audit.current_app_user()</code>
I12	Every change is logged with old and new values	trigger <code>audit.log_change</code>
I13	Nothing is hard-deleted by editors; deletion is admin-only, history kept in <code>audit.change_log</code>	GRANTS

Domain events (documented, not event-sourced)

`ImportFinished` → `job row + refresh_derived()` (`counts, bbox`) · `LayerValidated` · `TilesetBuilt` (`Ready`) · `LayerPublished` · `TilesetSwitched` (`v1→v2`) · `LayerArchived` .

Decisions that shape the model

- Layer is a **publishing slice**, not a partition of features: one CityGML building may carry LOD1, LOD2 and LOD4 geometry at once. Features belong to a dataset via the lineage tag.
- Version belongs to **tileset** (builds), not layer; feature history is already in 3DCityDB (`creation_date` , `termination_date`).
- Rules are in the database so every entry point (admin, API, script, `psql`) obeys them.